



Bilkent University

CS 319

Deliverable-1

S1-T2-Databaseable

Hüsnü Arkca

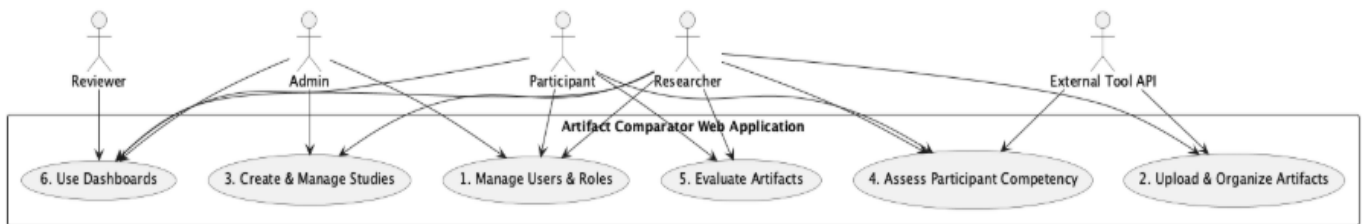
Yunus Emre Aslan

Mehmet Dizdar

Aral Müftüoğlu

Onur Ege Yakar

1. Summary Use Case Diagram

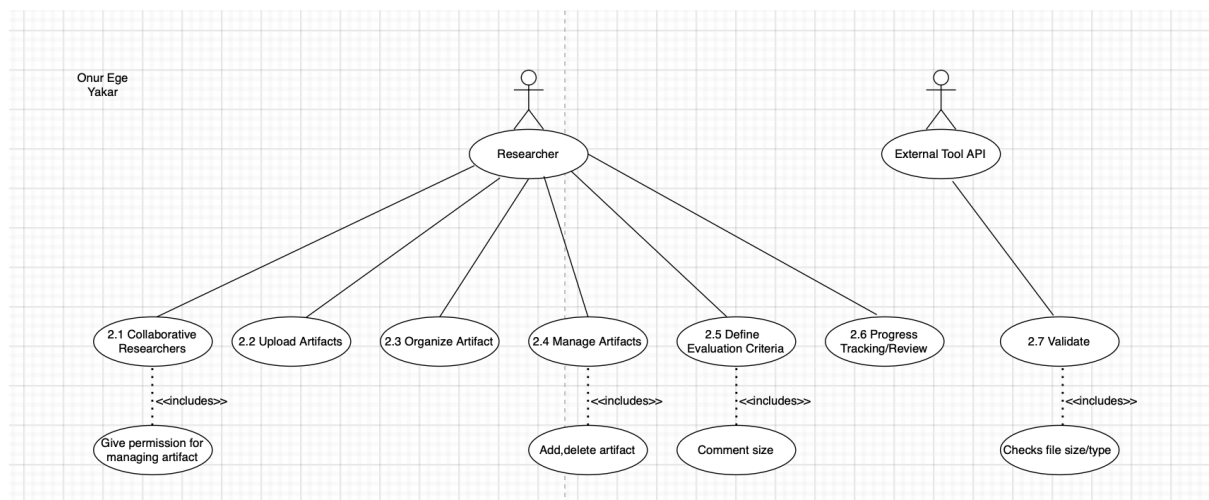


2. Level-1 Use Case Diagrams

Level-1 use case diagrams from each participant of the team:

2.1 Upload & Organize Artifacts

Created by: Onur Ege Yakar



Actors:

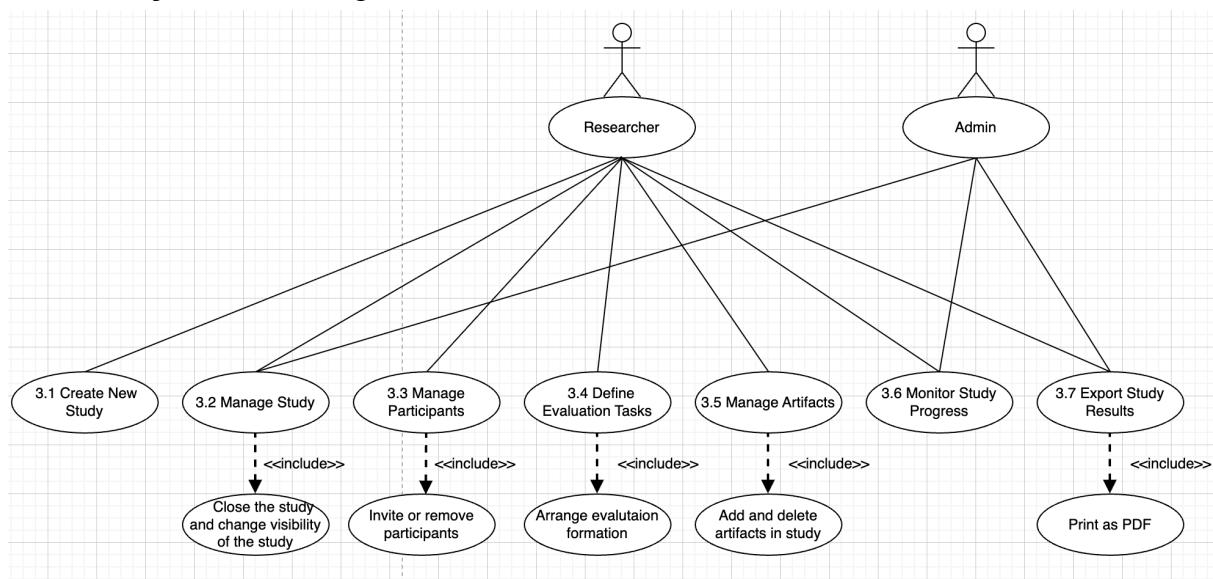
- Researcher
- External API Tool

Level-1 Use Cases:

- Collaborative Researchers
- Upload Artifacts
- Organize Artifacts
- Manage Artifacts
- Define Evaluation Criteria
- Progress Tracking
- Validate Artifacts

2.2 Create & Manage Studies

Created by: Aral Müftüoğlu



Actors:

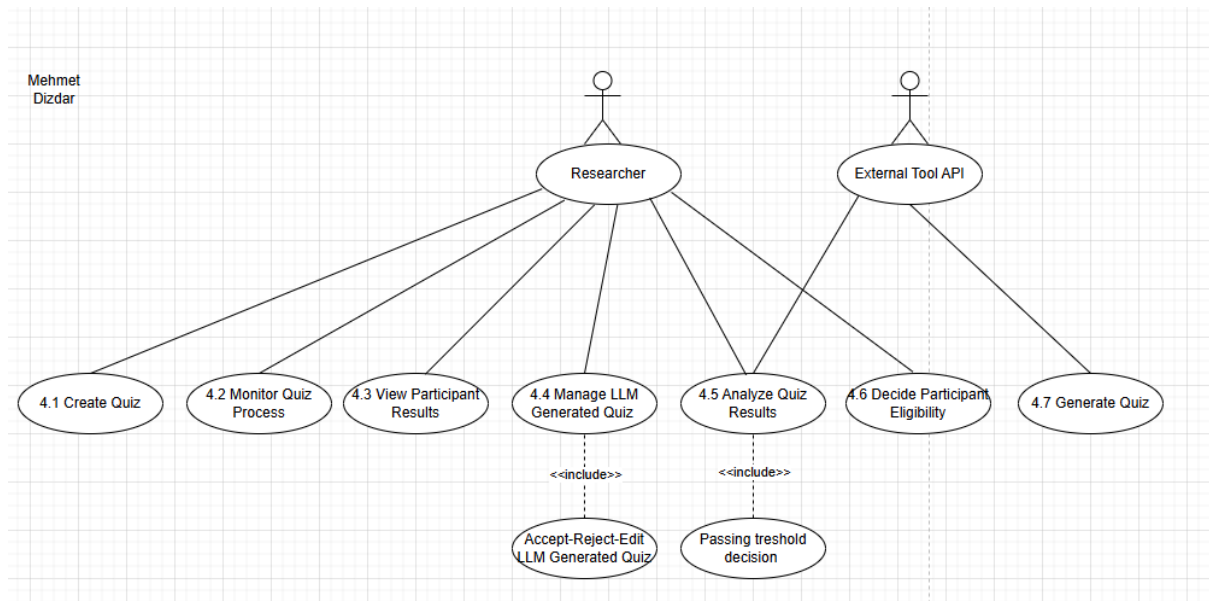
- Researcher
- Admin

Level-1 Use Cases:

- Create New Study
- Manage Study
- Manage Participants
- Define Evaluation Tasks
- Manage Artifacts
- Monitor Study Progress
- Export Study Results

2.3 Assess Participant Competency

Created by: Mehmet Dizdar



Actors:

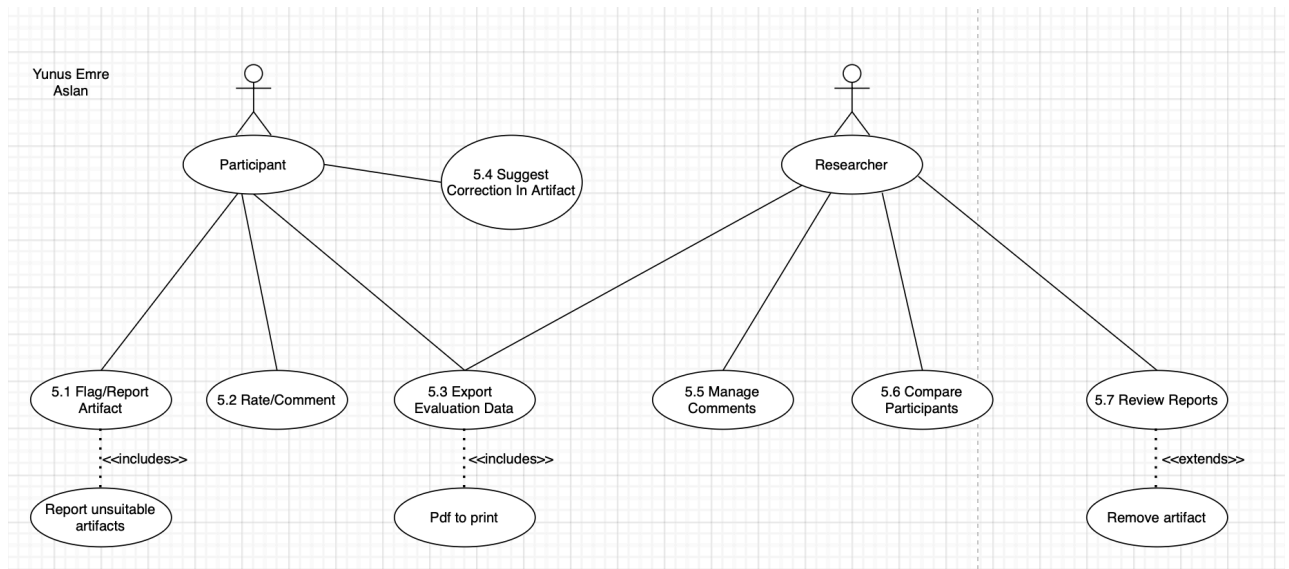
- Researcher
- External Tool API

Level-1 Use Cases:

- **Create Quiz**
- **Monitor Quiz Process**
- **View Participant Results**
- **Manage LLM Generated Quiz**
- **Analyze Quiz Results**
- **Decide Participant Eligibility**
- **Validate Artifacts**

2.4 Evaluate Artifacts

Created by: Yunus Emre Aslan



Actors:

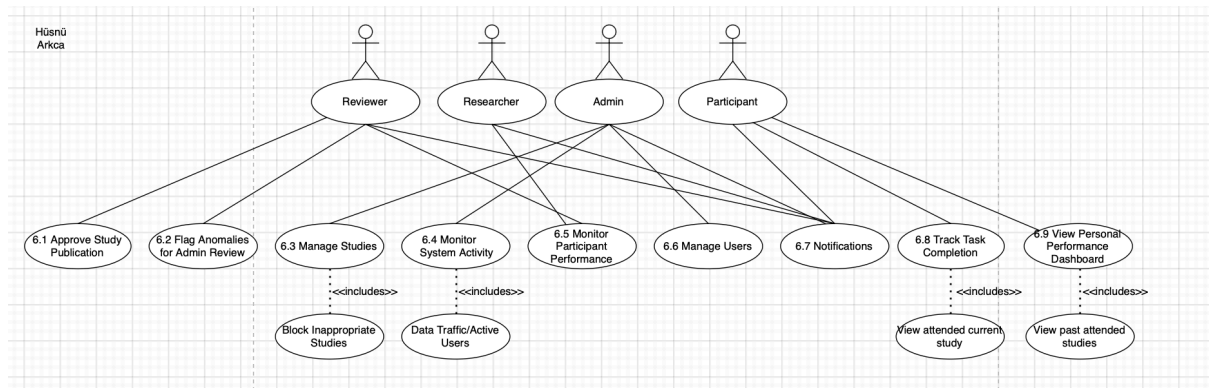
- **Researcher**
- **Participants**

Level-1 Use Cases:

- **Flag/Report Artifact**
- **Rate/Comment**
- **Export Evaluation Data**
- **Suggest Correction in Artifact**
- **Manage Comments**
- **Compare Participants**
- **Review Reports**

2.5 Use Dashboards

Created by: Hüsnü Arkca



Actors:

- **Researcher**
- **Reviewer**
- **Admin**
- **Participants**

Level-1 Use Cases:

- **Approve Study Publication**

- **Flag Anomalies for Admin Review**
- **Manage Studies**
- **Monitor System Activity**
- **Monitor Participant Performance**
- **Manage Users**
- **Notifications**
- **Track Test Compilation**
- **View Personal Performance Dashboard**

Tech Stack

Frontend Framework: React

→ We chose React because of its component-based architecture and large ecosystem of libraries and UI components.

Backend Framework: Spring Boot

→ Spring Boot provides a strong and secure foundation for backend development. It simplifies the process of building RESTful APIs, integrates simply with databases, and supports essential features such as dependency injection and security configuration.

In addition, all of us were also familiar with Java from previous courses and experienced with Spring Boot from our internships. So we chose it to have a quick and strong start for backend development.

Database: PostgreSQL

→ We chose PostgreSQL because it is a reliable and compact open-source database that handles complex data efficiently. It supports advanced SQL queries, relationships between tables, and large amounts of data with high performance.

Deployment Tools: Docker

→ We chose Docker because it simplifies deployment, reduces configuration errors, and allows for easy maintenance. Additionally, Docker eliminates the need for each team member to manually download and configure tools such as PostgreSQL or the

application server, as all required services are packaged within containers. This approach ensures that the entire development team operates in an identical, pre-configured environment reducing setup time.

Potential Integrations: Gemini

→ We plan to integrate Google Gemini to enhance the system with AI-driven capabilities such as intelligent data analysis, report generation, and natural language interaction. Gemini's multimodal capabilities—understanding text, images, and structured data—make it suitable for future expansion toward features like generating quizzes, and analyzing results.

Non-Functional Requirements

- 1) **Security:** The system will enforce role-based access control to restrict access to administrative features from users and research data from participants.
- 2) **Security:** The system will encrypt both passwords and email addresses to provide safety of user data.
- 3) **Reliability:** The application will maintain 90% uptime per month.
- 4) **Usability:** The application will allow participants to complete artifact comparison and annotation tasks without training. The system shall make $\geq 90\%$ of users complete comparison tasks successfully in usability tests.
- 5) **Performance:** The system will respond to user requests within 5 seconds under 100 active users.
- 6) **Compatibility:** The system will be available in mobile devices.